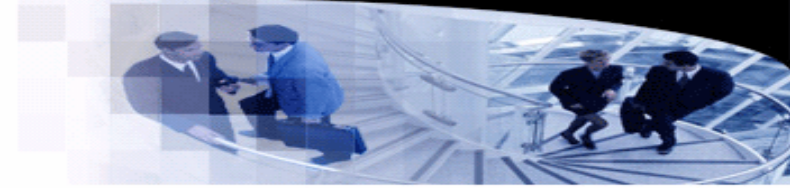




武汉大学
WUHAN UNIVERSITY



Programming Languages and Compilers

Lecture 17. Intermediate Representations

袁梦霆 (YUAN Mengting)

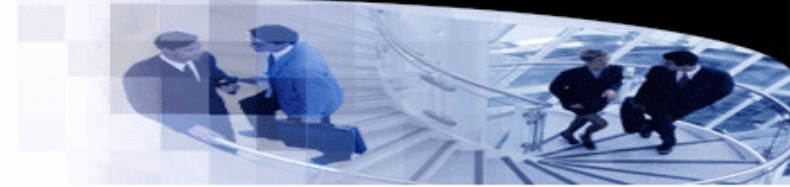
ymt@whu.edu.cn

School of Computer Science, Wuhan University

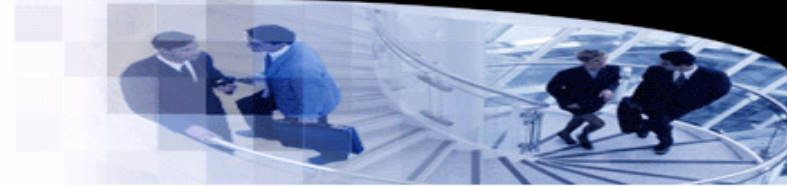
Motivation

- A compiler for arithmetic expressions
 - Example:

Productions	Semantic Rules	Semantic Actions
$E' \rightarrow E$		<code>{print_code \$\$code;}</code>
$E \rightarrow E^1 + E^2$	$E.value = E^1.value + E^2.value$	<code>{\$\$code := \$1code; \$\$code += "mv a2,a1\n"; \$\$code += \$3code; \$\$code += "addi a1,a1,a2\n";}</code>
$E \rightarrow E^1 * E^2$	$E.value = T.value$	<code>{\$\$code := \$1code; \$\$code += "mv a2,a1\n"; \$\$code += \$3code; \$\$code += "mul a1,a1,a2\n";}</code>
$E \rightarrow (E^1)$	$T.value = T^1.value * F.value$	<code>{\$\$code := \$2code;}</code>
$E \rightarrow id$		<code>{\$\$code := "lw a1," + id.text + "\n";}</code>



Intermediate Representation



- **Intermediate representation (IR)**

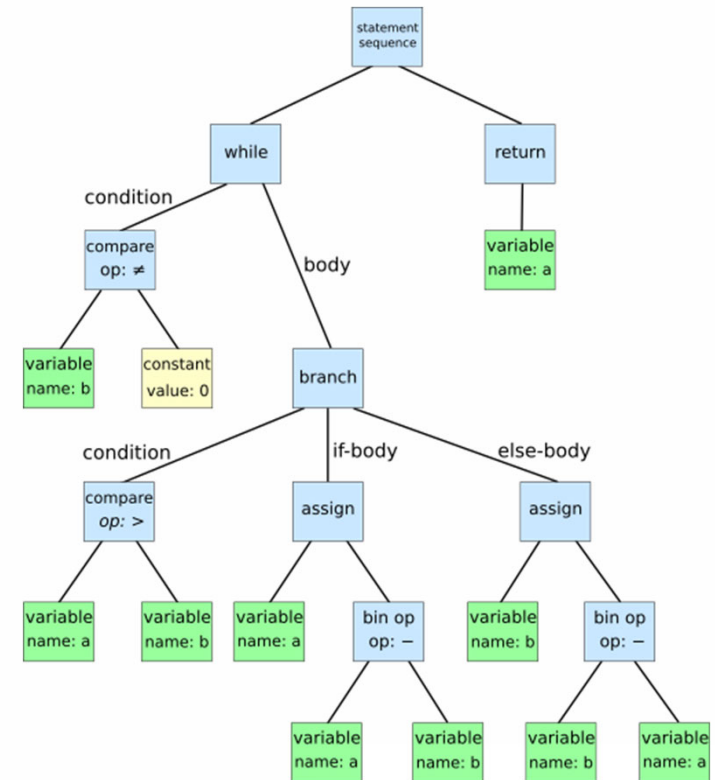
- There are many transformations/optimization algorithms inside a compiler.
- These transformations require different kinds of information.
- The data structures of these information are called intermediate representations (IRs).
- Each kind of IR is suitable of representing some specific information of programs.
 - Trees are good at describing hierarchy relationships in programs.
 - Sequential representations can specify temporal orders of instructions.
- Commonly used IRs:
 - Abstract syntax tree (AST), Quadruple, Directed acyclic graph (DAG), Control-flow graph (CFG), Data-flow graph
- Intermediate languages:
 - Register transfer language (RTL), Java bytecode, LLVM IR, Microsoft Common Intermediate Language for .NET

Intermediate Representation

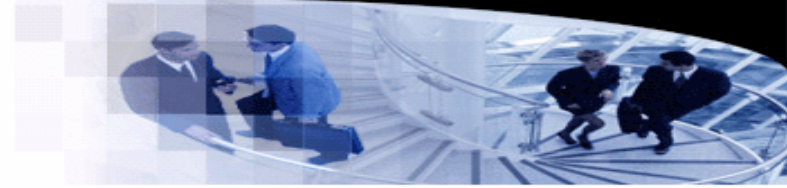
- Abstract syntax tree (AST)

- A tree-style data structure to represent the structure of a program.
- Each node is either an operator or an operand.
- ASTs are often generated by parsers.
- They are then used to generate other IRs.

```
while b ≠ 0:  
    if a > b:  
        a := a - b  
    else:  
        b := b - a  
return a
```



Intermediate Representation



- **Quadruple**

- A 4-tuple representation.
- A quadruple is in the form of $x := y \text{ op } z$, where x , y , and z are variables and op is the operator.
- RISC-V instructions are actually quadruples.

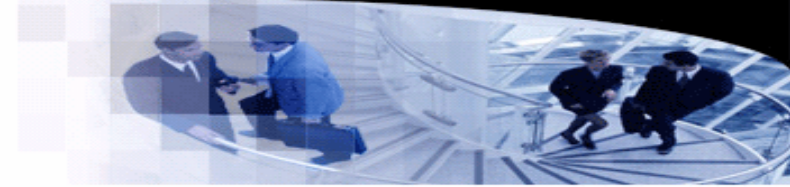
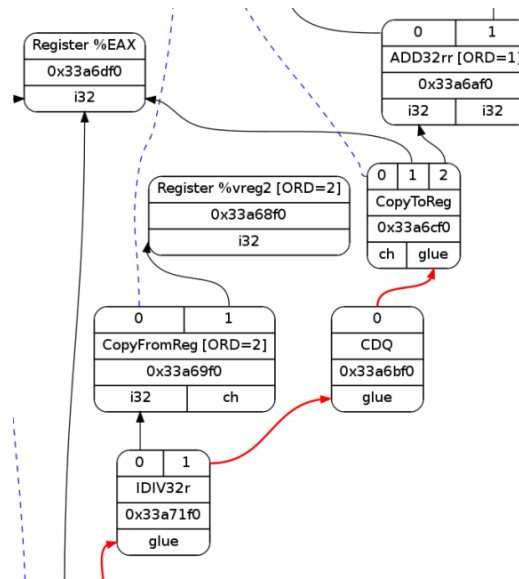
```
int bar(int x, int y) {  
    return x * y;  
}
```

```
define dso_local noundef i32 @_Z3barii(i32 noundef %0, i32 noundef %1)  
    %3 = alloca i32, align 4  
    %4 = alloca i32, align 4  
    store i32 %0, i32* %3, align 4  
    store i32 %1, i32* %4, align 4  
    %5 = load i32, i32* %3, align 4  
    %6 = load i32, i32* %4, align 4  
    %7 = mul nsw i32 %5, %6  
    ret i32 %7  
}
```

Intermediate Representation

- **Directed acyclic graph (DAG)**

- A directed acyclic graph (DAG) is a directed graph with no directed cycles.
- Compared to ASTs, DAGs reuse nodes.
- DAGs may contain data flows and control flows.



Intermediate Representation

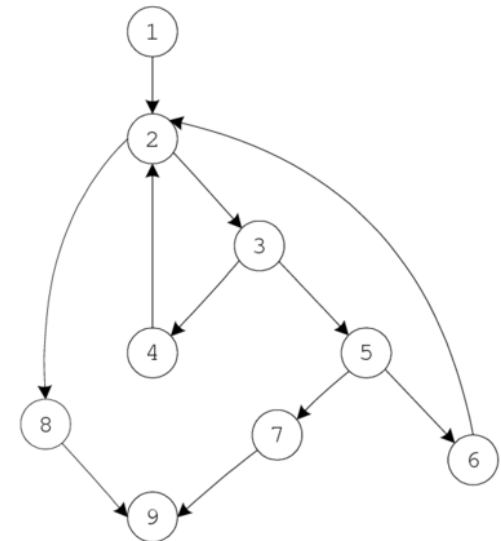
- Control-flow graph (CFG)

- A graph to represent control-flow information of a program.
- Each node represents a **basic block**.
- A basic block is a sequence of instructions with only a single entry point and a single exit point, where no branches or jumps occur within the block.
- An directed edge (b_1, b_2) represents a branch from b_1 to b_2 , indicating that b_2 may be executed after b_1 .

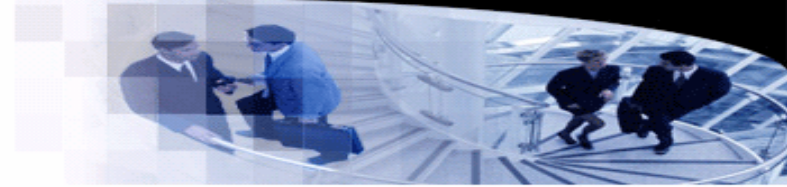
Source Program:

```
int binsearch(int x, int v[], int n)
{
    int low, high, mid;
1 | low = 0;
  | high = n - 1;
  | while (low <= high) | 2
  | {
    | 3 | mid = (low + high)/2;
    |   | if (x < v[mid])
    |   |   high = mid - 1; | 4
    |   | 5 | else if (x > v[mid])
    |   |   low = mid + 1; | 6
    |   | 7 | else return mid;
    |   | }
    | 8 | return -1; | 8
  | 9 }
```

CFG:



Intermediate Representation

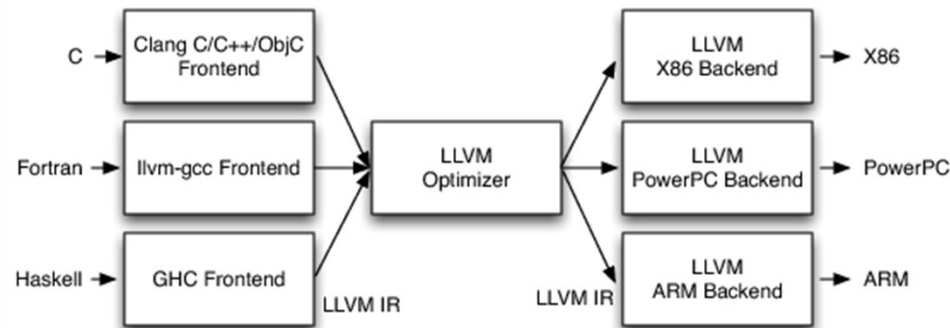


- **Intermediate language**

- Intermediate languages are intermediate representations that targets a specific virtual machine.
- Intermediate languages can be used to write programs that run on a virtual machine.

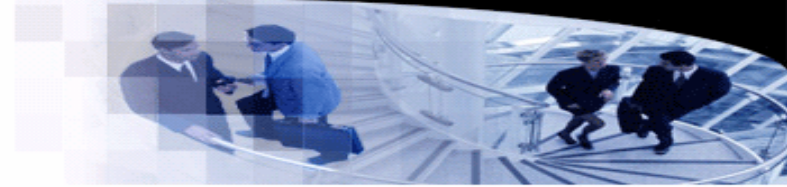
- **LLVM language**

- The LLVM language (LLVM IR) is the code representation for low-level virtual machine (LLVM).



- The LLVM code representation is designed to be used in three different forms: as an in-memory compiler IR, as an bitcode representation (suitable for fast loading by a Just-In-Time compiler), and as a human readable assembly language representation.

Intermediate Representation



- LLVM language

- LLVM programs are composed of Module's.
- A Module is a translation unit that consists of functions, global variables, and symbol table entries.

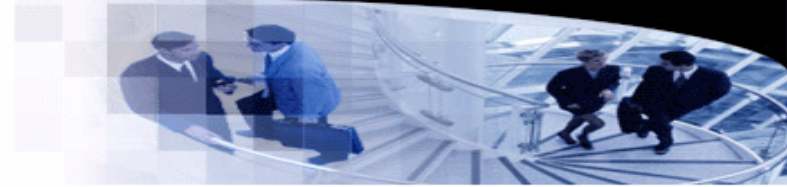
```
; Declare the string constant as a global constant.
@.str = private unnamed_addr constant [13 x i8] c"hello world\0A\00"

; External declaration of the puts function
declare i32 @puts(ptr captures(none)) nounwind

; Definition of main function
define i32 @main() {
    ; Call puts function to write out the string to stdout.
    call i32 @puts(ptr @.str)
    ret i32 0
}

; Named metadata
!0 = !{i32 42, null, !"string"}
!foo = !{!0}
```

Intermediate Representation



- LLVM language

Global variable.

- LLVM programs are composed of Module's.
- A Module is a translation unit that consists of functions, global variables, and symbol table entries.

```
; Declare the string constant as a global constant.
@.str = private unnamed_addr constant [13 x i8] c"hello world\0A\00"

; External declaration of the puts function
declare i32 @puts(ptr captures(none)) nounwind

; Definition of main function
define i32 @main() {
    ; Call puts function to write out the string to stdout.
    call i32 @puts(ptr @.str)
    ret i32 0
}

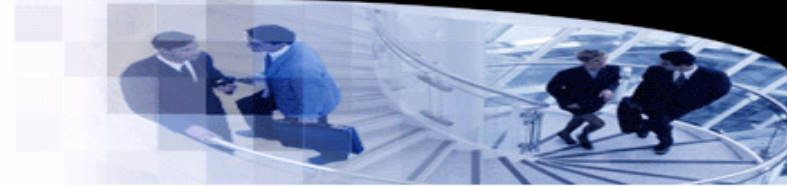
; Named metadata
!0 = !{i32 42, null, !"string"}
!foo = !{!0}
```

Function.

Instruction.

Type.

Intermediate Representation



- LLVM language
 - LLVM type system
 - Void type
 - Function type

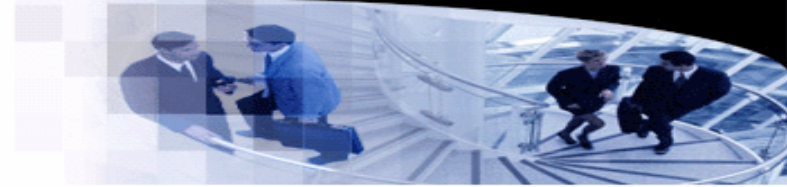
Syntax:

```
<returntype> (<parameter list>)
```

Examples:

<code>i32 (i32)</code>	function taking an <code>i32</code> , returning an <code>i32</code>
<code>i32 (ptr, ...)</code>	A vararg function that takes at least one pointer argument and returns an integer. This is the signature for <code>printf</code> in LLVM.
<code>{i32, i32} (i32)</code>	A function taking an <code>i32</code> , returning a structure containing two <code>i32</code> values

Intermediate Representation



- LLVM language
 - LLVM type system
 - First class types
 - Integer type

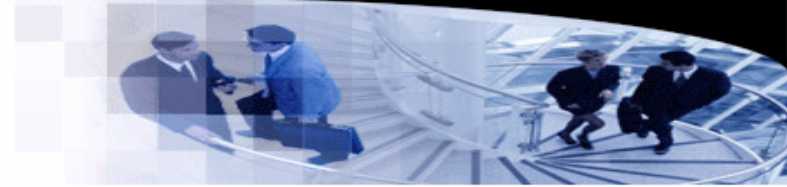
Syntax:

iN

Examples: ¶

i1	a single-bit integer.
i32	a 32-bit integer.
i1942652	a really big integer of over 1 million bits.

Intermediate Representation



- LLVM language
 - LLVM type system
 - First class types
 - Floating-point type

Type	Description
half	16-bit floating-point value (IEEE-754 binary16)
bfloat	16-bit “brain” floating-point value (7-bit significand). Provides the same number of exponent bits as float, so that it matches its dynamic range, but with greatly reduced precision. Used in Intel’s AVX-512 BF16 extensions and Arm’s ARMv8.6-A extensions, among others.
float	32-bit floating-point value (IEEE-754 binary32)
double	64-bit floating-point value (IEEE-754 binary64)
fp128	128-bit floating-point value (IEEE-754 binary128)
x86_fp80	80-bit floating-point value (x87)
ppc_fp128	128-bit floating-point value (two 64-bits)

- There are also pointer type, vector type, label type, token type, structure type, etc.